

✓ Word2Vec

Word2Vec: 단어 간의 유사성을 측정하기 위해 분포 가설을 기반으로 개발

- 분포 가설: 같은 문맥에서 함께 자주 나타나는 단어들을 서로 유사한 의미를 가질 가능성이 높다는 가정
 - 단어 간의 동시 발생 확률 분포를 이용해 단어 간의 유사성 측정
 - 분산 표현: 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것
 - 유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 가짐
- 룩업 연산: 이산 변수의 값을 연속적인 벡터로 변환하는 과정
- 임베딩의 유사도를 측정할 때는 코사인 유사도를 일반적으로 사용
 - 두 벡터 간의 각도를 이용하여 유사도 계산
- 젠심 라이브러리: 대용량 텍스트 데이터의 처리를 위한 메모리 효율적인 방법 제공
 - 학습된 모델 저장 및 관리, 유사도와 관련된 기능 제공
 - Word2Vec 클래스: 여러 하이퍼파라미터를 받아 쉽고 빠르게 모델 학습

단어 벡터화: 희소 표현, 밀집 표현

- 희소 표현: 원-핫 인코딩, TF-IDF 등(빈도 기반 방법)
 - 대부분의 벡터 요소가 0으로 표현
 - 단어 사전의 크기가 커지면 공간적 낭비 발생, 단어 간의 유사성 반영 X
- 밀집 표현: Word2Vec
 - 고정된 크기의 실수 벡터로 표현, 벡터 공간상에서 단어 간의 거리를 효과적으로 계산
 - 단어 임베딩 벡터: 밀집 표현된 벡터

CBoW: 주변에 있는 단어를 가지고 중간에 있는 단어를 예측

- 중심 단어: 예측해야 할 단어 / 주변 단어: 예측에 사용되는 단어들
- 윈도우: 중심 단어를 예측하기 위해 몇 개의 주변 단위를 고려할지에 대한 범위
- 슬라이딩 윈도우: 윈도우를 이동해 가며 학습하는 것, CBoW가 사용
- CBoW 모델 학습 과정:
 1. 입력 단어의 원-핫 벡터를 투사층의 입력값으로 받음
 - 투사층: 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표 구조
 2. 투사층 통과, 각 단어는 E 크기의 임베딩 벡터로 변환
 3. 임베딩 벡터의 평균값 계산
 4. 계산된 평균 벡터를 가중치 행렬과 곱하여 V 크기의 벡터를 얻음
 5. 소프트맥스 함수를 이용해 중심 단어 예측

- 하나의 윈도우에서 하나의 학습 데이터가 만들어짐

Skip-gram: 중심 단어를 입력으로 받아서 주변 단어를 예측

- 중심 단어를 기준으로 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼음
- 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터가 만들어짐

계층적 소프트맥스: 출력층을 이진 트리 구조로 표현해 연산을 수행

- 자주 등장하는 단어일수록 상위 노드에 위치
- 각 노드는 학습이 가능한 벡터를 가짐, 입력값은 해당 노드의 벡터와 내적값 계산 후 시그모이드 함수로 확률 계산
- 잎 노드: 가장 깊은 노드, 각 단어를 의미
 - 각 단어의 확률은 경로 노드의 확률을 곱해서 구할 수 있음
- $O(\log 2V)$ 의 시간 복잡도를 가짐

네거티브 샘플링: 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용

- 학습 윈도우 내에 등장하지 않는 단어를 n 개 추출하여 정답 단어와 함께 소프트맥스 연산 수행
- 네거티브 샘플링의 추출 확률:

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

- 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류를 함
 - 로지스틱 회귀 모델 사용
- 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치를 학습
- 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블로 내적 연산 수행

임베딩 클래스

- num_embeddings: 임베딩 수, 단어 사전의 크기
- embedding_dim: 임베딩 차원, 임베딩 벡터의 크기
- padding_idx: 패딩 인덱스, 패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터를 0으로 설정
- norm_type: 임베딩 벡터의 크기를 제한하는 방법을 선택
- max_norm: 임베딩 벡터의 최대 크기 지정

Word2Vec 클래스

- sentences: 모델의 학습 데이터
- corpus_file: 학습 데이터를 파일로 입력할 때 파일의 경로
- vector_size: 학습할 임베딩 벡터의 크기
- alpha: 모델의 학습률
- window: 학습 데이터를 생성할 윈도우의 크기
- min_count: 학습에 사용할 단어의 최소 빈도
- max_final_vocab: 단어 사전의 최대 크기
- workers: 빠른 학습을 위해 병렬 학습할 스레드의 수
- Skip-gram(sg): Skip-gram 모델의 사용 여부
- 계층적 소프트맥스(hs): 계층적 소프트맥스 사용 여부 cbow_mean: CBoW 모델로 구성할 때 사용되는 하이퍼파라미터로 중심 단어와 주변 단어를 합쳐서 하나의 벡터로 만들 때, 합한 벡터의 평균화 여부
- negative: 네거티브 샘플링의 단어 수
- ns_exponent: 네거티브 샘플링 확률의 지수
- epochs: 학습 에폭 수
- batch_words: 몇 개의 단어로 학습 배치를 구성할지

✓ fastText

fastText: 텍스트 분류 및 텍스트 마이닝을 위한 알고리즘, 단어와 문장을 벡터로 변환하는 기술 기반

- 하위 단어 고려, N-gram을 사용해 단어를 분해하고 벡터화
- 단어의 벡터화를 위해 특수 기호 사용, 기호가 추가된 단어는 하위 단어 집합으로 분해
- 다양한 N-gram을 적용해 입력 토큰을 분해하고 하위 단어 벡터를 구성함으로써 단어의 부분 문자열을 고려하는 하위 단어 집합 생성
- OOV 단어도 하위 단어로 나누어 임베딩 계산 가능
- CBoW와 Skip-gram으로 구성, 네거티브 샘플링 기법을 사용해 학습

fastText 클래스

- 대부분 Word2Vec와 동일
- min_n: N-gram의 최소값
- max_n: N-gram의 최대값

KorNLI 데이터세트: 한국어 자연어 추론을 위한 데이터 세트

- 주어진 문장이 함의 관계, 독립 관계, 불일치 관계 중 어느 관계에 해당하는지 분류

✓ 순환 신경망

순환 신경망: 순서가 있는 연속적인 데이터를 처리

- 각 시점의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동
- 자연어 데이터: 연속적인 데이터의 일종, 문장 안에서 단어들이 순서대로 등장
 - 한 단어가 이전 단어들과 상호작용하여 문맥으로 이루고 의미를 형성
 - 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 상관관계 존재
 - 시계열 데이터, 자연어 처리, 음성 인식 및 기타 시퀀스 데이터와 같은 도메인에서 널리 사용
- 연속형 데이터를 순서대로 입력받아 처리, 각 시점마다 은닉 상태의 형태로 저장
 - 셀: 은닉 상태와 출력값을 계산하는 노드

수식 6.11 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

수식 6.12 순환 신경망의 출력값

$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

일대다 구조: 하나의 입력 시퀀스에 대해 여러 개의 출력값 생성

- 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 이미지 캡셔닝 모델이 됨
- 구현을 위해서는 출력 시퀀스의 길이를 미리 알고 있어야 함

다대일 구조: 여러 개의 입력 시퀀스에 대해 하나의 출력값 생성

- 문장 분류, 두 문장 간의 관계를 추론하는 자연어 추론 등에 적용

다대다 구조: 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용

- 입력 시퀀스의 길이와 출력 시퀀스의 길이는 서로 다를 수 있음
- 시퀀스-시퀀스 구조
 - 입력 시퀀스를 처리하는 인코더와 출력 시퀀스를 생성하는 디코더로 구성

양방향 순환 신경망: 기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안

- 이전 시점의 은닉 상태 뿐만 아니라 이후 시점의 은닉 상태도 이용

다중 순환 신경망: 여러 개의 순환 신경망을 연결

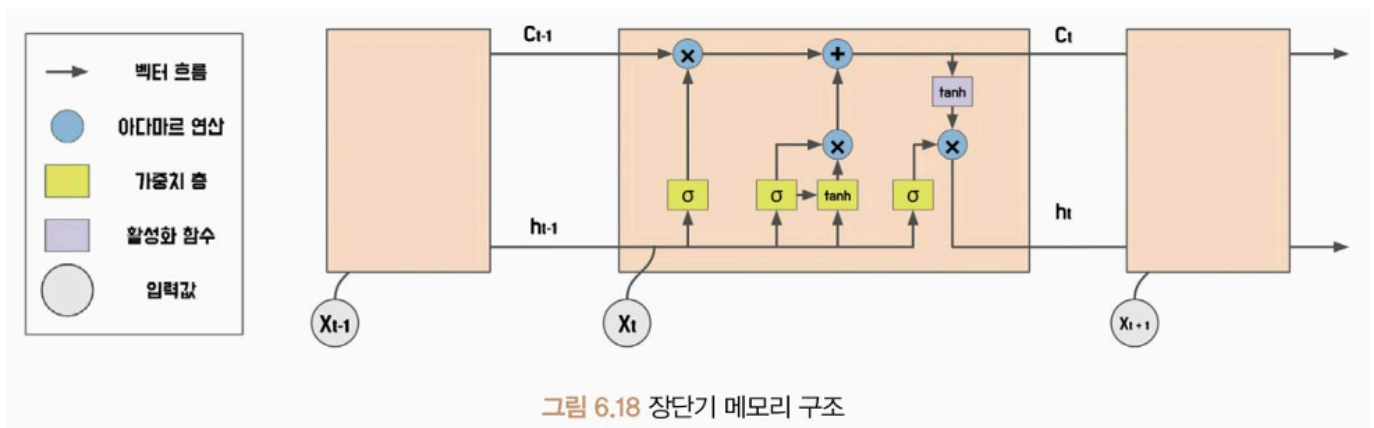
- 여러 개의 순환 신경망 층으로 구성, 각 층에서의 출력값은 다음 층으로 전달되어 처리
- 다양한 특징을 처리할 수 있어 성능이 향상, 층이 깊어질수록 더 복잡한 패턴 학습 가능
- 그러나 학습 시간이 오래 걸리고, 기울기 소멸 문제가 발생할 수 있음!

순환 신경망 클래스

- input_size와 hidden_size를 입력해 순환 신경망 구성
- num_layers: 순환 신경망의 층 수
- nonlinearity: 활성화 함수의 종류
- bias: 평향 값 유무
- batch_first: 입력 배치 크기를 첫 번째 차원으로 사용할지 여부
- dropout: 과대적합 방지를 위한 드롭아웃 확률 설정
- bidirectional: 순환 신경망 구조가 양방향으로 처리할지 여부

장단기 메모리: 기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결

- 기존 순환 신경망은 장기 의존성 문제가 발생할 수 있음
- 메모리 셀과 게이트 구조 도입



- 셀 상태와 망각 게이트, 기억 게이트, 출력 게이트로 정보의 흐름 제어
 - 셀 상태: 정보를 저장하고 유지하는 메모리 역할

- 망각 게이트: 이전 셀 상태에서 어떠한 정보를 삭제할지 결정
- 입력 게이트: 새로운 정보를 어떤 부분에 추가할지 결정
- 출력 게이트: 셀 상태의 정보 중 어떤 부분을 출력할지 결정
- 메모리 셀: 현재 시점의 입력과 이전 시점의 은닉 상태를 기반으로 정보를 계산하고 저장
 - 출력값 계산에 직접 사용은 X
- 망각, 입력, 출력 게이트 모두 활성화 함수로 시그모이드 사용
 - 망각 게이트: 현재 시점의 입력과 이전 시점의 은닉 상태를 입력으로 받아 시그모이드 함수를 거친 값과 메모리 셀을 곱한 값으로 이전 시점의 메모리 셀을 업데이트
 - 기억 게이트: 시그모이드 함수를 거친 값과 하이퍼볼릭 탄젠트 함수를 거친 값의 곱으로 새로운 기억 값 계산
 - 출력 게이트: 현재 시점의 입력과 이전 시점의 은닉 상태, 그리고 새로 업데이트된 메모리 셀을 입력으로 받아 현재 시점의 출력값을 계산

수식 6.13 망각 게이트

$$f_t = \sigma(W_x^{(f)}x_t + W_h^{(f)}h_{t-1} + b^{(f)})$$

수식 6.14 기억 게이트

$$g_i = \tanh(W_x^{(g)}x_t + W_h^{(g)}h_{t-1} + b^{(g)})$$

$$i_i = \text{sigmoid}(W_x^{(i)}x_t + W_h^{(i)}h_{t-1} + b^{(i)})$$

수식 6.15 메모리 셀

$$c_t = f_t \odot c_{t-1} + g_i \odot i_t$$

- 아다마르 곱: 원소 별 곱셈 연산

수식 6.16 출력 게이트

$$o_t = \sigma(W_x^{(o)}x_t + W_h^{(o)}h_{t-1} + b^{(o)})$$

수식 6.17 은닉 상태 갱신

$$h_t = o_t \odot \tanh(c_t)$$

장단기 메모리 클래스

- 순환 신경망 클래스와 거의 유사, nonlinearity는 사용 X
- proj_size: 장단기 메모리 계층의 출력에 대한 선형 투사 크기를 결정
 - 투사 크기가 0보다 큰 경우, 은닉 상태를 선형 투사를 통해 다른 차원으로 매핑
 - 투사 크기는 은닉 상태보다 작은 값으로 설정

긍/부정 분류 모델 구조

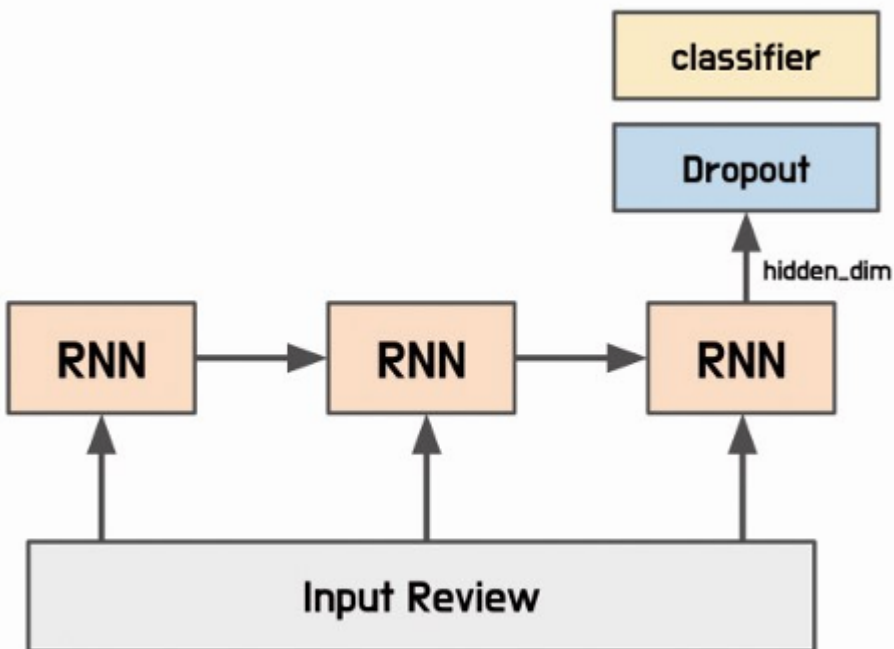


그림 6.23 긍/부정 분류 모델 구조

- 임베딩 계층: 입력 텍스트를 정수 인코딩한 뒤 해당 색인 값에 해당하는 임베딩을 가져옴
- 학습 데이터세트와 테스트 데이터세트를 정수로 인코딩하면 모델이 텍스트 데이터를 처리하기 쉬워짐
- RMSProp: 모든 기울기를 누적하지 않고, 지수 가중 이동 평균을 사용해 학습률 조절

✓ 합성곱 신경망

합성곱 신경망: 주로 이미지 인식과 같은 컴퓨터비전 분야의 데이터를 분석하기 위해 사용

- 합성곱 연산: 이미지의 특정 영역에서 입력값의 분포 또는 변화량을 계산해 출력 노드를 생성
 - 특정 영역 안에서 연산 수행, 지역 특징을 효과적으로 추출
- 지역 특징을 조합에 입력 데이터의 전반적인 전역 특징 파악
- 입력 데이터의 길이에 상관없이 병렬 처리 가능, 학습에 필요한 가중치 수를 줄여 깊은 신경망 구성
- Adam 최적화 함수 사용
 - 이전 기울기 값의 지수 이동 평균과 지수 이동 제곱 평균을 사용해 현재 기울기 값 갱신

합성곱 계층: 입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층

- 데이터의 지역적인 패턴 인식, 모델 매개변수 공유
 - 모델 매개변수를 공유함으로써 과대적합 방지
 - 필터를 사용해 특징을 추출하므로 특징이 어디에 있어도 동일하게 추출
- 필터: 커널, 윈도우라고도 부름
 - 일정 간격으로 이동하면서 입력 데이터와 합성곱 연산을 수행해 특징 맵 생성
 - 영역마다 합성곱 연산 수행, 가중치가 모델 학습 과정에서 갱신
 - 합성곱 신경망은 보통 여러 개의 필터를 사용함
- 패딩: 입력 이미지나 입력으로 사용되는 특징 맵 가장자리에 특정 값을 덧붙임
 - 패딩 값을 0으로 할당하면 제로 패딩
- 간격: 필터가 한 번에 움직이는 크기
 - 간격의 크기를 조절함으로써 출력 데이터의 크기 조절
 - 입력 데이터의 공간적인 정보 유지 혹은 감소 가능
- 채널: 입력 데이터와 필터 간의 연산 수행
 - 입력 데이터와 필터가 3차원으로 구성되어 있을 때 같은 위치의 값끼리 연산되게 함
 - 개수는 일반적으로 합성곱 계층에서 설정
 - 개수가 많아질수록 학습할 수 있는 특징의 다양성이 증가, 모델의 표현력이 높아짐
- 팽창: 합성곱 연산을 수행할 때 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법
 - 필터와 입력 데이터 사이에 간격을 두는 방법
 - 값이 커질수록 필터가 입력 데이터를 바라보는 범위가 넓어짐
 - 적절히 적용하면 모델의 매개변수 수를 줄일 수 있어 메모리 사용량 줄이기 가능

합성곱 계층 클래스

- in_channels, out_channels로 채널을 생성
- kernel_size: 합성곱 연산의 필터 크기
- stride, padding, dilation: 필터의 속성
- groups: 입력 채널과 출력 채널을 하나의 그룹으로 묶는 것
- bias: 편향 값 포함 여부
- padding_mode: 패딩 영역에 할당되는 값 설정

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

활성화 맵: 합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값

- 합성곱 연산과 활성화 함수를 여러 번 반복하여 신경망 구성
- 활성화 함수를 적용함으로써 모델이 비선형성을 가지게 되어 입력 데이터에서 다양한 추상적 특징 학습 가능
- 합성곱 모델을 분석하는 데 매우 중요한 정보를 제공

풀링: 특징 맵의 크기를 줄이는 연산, 합성곱 계층 다음에 적용

- 일정한 크기의 필터 내 특정 값을 선택
 - 최댓값 풀링 / 평균값 풀링
 - 최댓값 풀링: 특정 크기의 필터 내 원소 값 중 가장 큰 값 선택
 - 평균값 풀링: 필터 내 원소 값의 평균값으로 특징 맵의 크기 감소

풀링 클래스

- 2차원 평균값 풀링 클래스는 팽창 지원 X
- count_include_pad: 패딩 영역의 값을 평균 계산에 포함할지 여부를 설정

수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

완전 연결 계층: 각 입력 노드가 모든 출력 노드와 연결된 상태

- 출력 노드 수 조절 가능, 모델의 복잡성과 용량 조절
- 이전 계층에서 추출된 특징을 촬영하여 최종적인 분류 작업 수행
 - 최종값은 소프트맥스나 시그모이드와 같은 활성화 함수를 적용해 분류 모델로 구성
 - 합성곱 신경망의 최종 출력을 결정하는 역할

텍스트 데이터는 1차원 합성곱을 적용해야 함

- 문장을 단어 단위로 분리하여 각 단어를 임베딩하여 나온 1차원 벡터 데이터를 입력값으로 사용

- 수평 방향으로 이동하지 않고 수직 방향으로만 이동하는 필터 적용
- 1차원 필터의 크기는 필터의 높이에만 영향, 너비는 입력 임베딩의 크기
- 다양한 크기의 합성곱 필터를 사용하여 여러 종류의 정보 추출 가능
 - 1차원 벡터를 출력값으로 얻게 되므로 풀링 적용 시 하나의 스칼라값 도출